

# Solution Architecture

## 1. The design claim

---

**The LLM has capability; the layers below it have authority.**

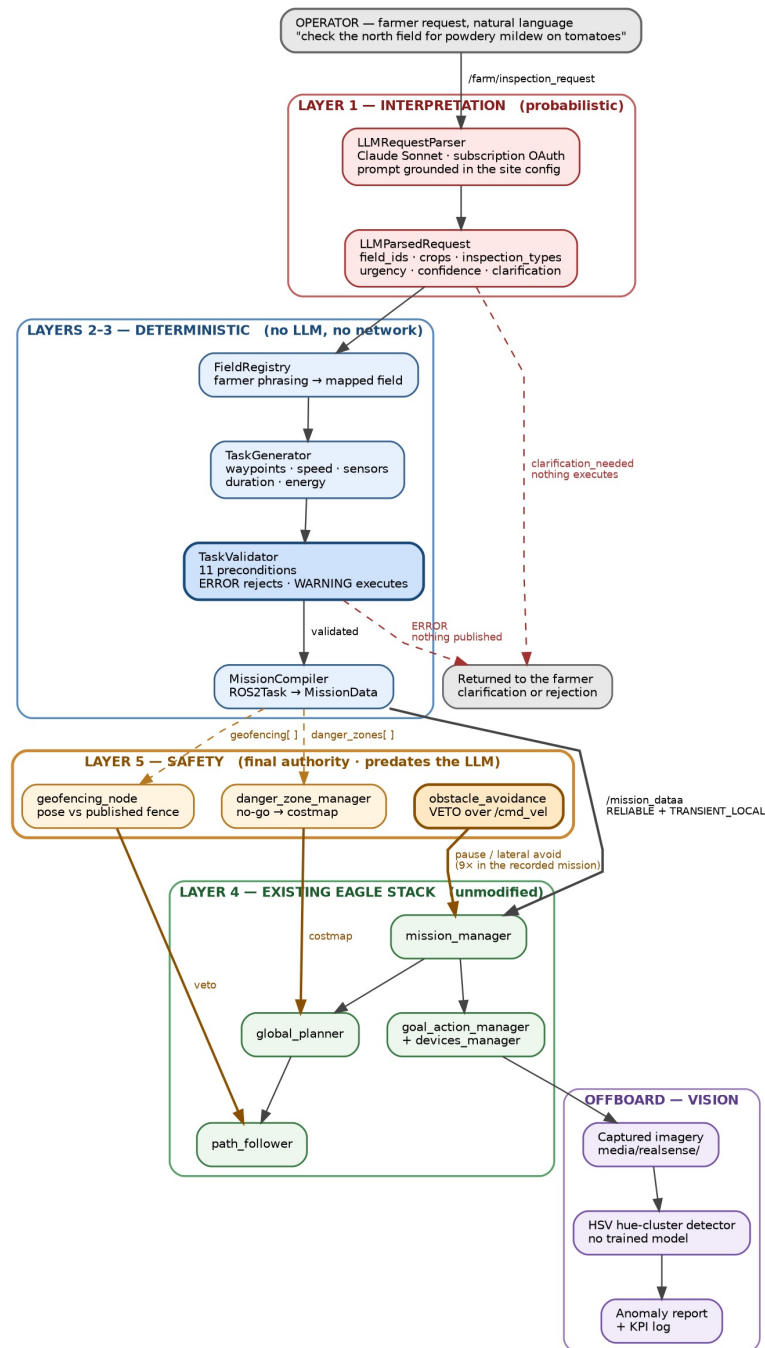
The model never emits a pose, a velocity or a ROS message. It returns a parsed request — a small JSON object naming fields, crops and inspection types. Every stage after that is deterministic code. By the time anything reaches the robot, it is the same `navigation_interfaces/MissionData` a human operator produces from the RViz toolbar, and the safety nodes that supervise it were built and tested before the LLM existed.

Everything below follows from that decision.

## 2. Block diagram

---

Editable sources: `architecture.dot` (Graphviz, used to render the figure below) and `architecture.mmd` (Mermaid, equivalent). Regenerate with `dot -Tpng -Gdpi=150 architecture.dot -o architecture.png`.



Read top to bottom: the operator's sentence enters at the top; everything red is probabilistic; everything blue is deterministic code; green is your existing stack, unmodified; orange is the safety layer, which has arrows *into* the execution stack and none coming back from the planner. The two dashed red arrows are the paths where nothing reaches the robot at all.

### 3. Mission flow, worked example

| Stage                                             | Output                                                                  | Measured |
|---------------------------------------------------|-------------------------------------------------------------------------|----------|
| Operator publishes to<br>/farm/inspection_request | "check the north field for powdery mildew on tomatoes"                  | —        |
| LLM parses                                        | north_field ·<br>tomato ·<br>disease_detection ·<br>routine · conf 0.95 | 3.7 s    |
| Field resolution                                  | north_field → rows<br>north_1, north_2                                  | —        |

|                   |                                                            |        |
|-------------------|------------------------------------------------------------|--------|
| Task generation   | 14 waypoints, 0.3 m/s, 109.0 m, 38.2 Wh, 9 min             | —      |
| <b>Validation</b> | 11 preconditions pass                                      | —      |
| Compilation       | 15 goals (14 + return to base), 1 geofence, 2 danger zones | —      |
| Execution         | 15/15 REACHED, mission completed                           | ~9 min |
| Safety            | 9 lateral-avoidance interventions, all auto-recovered      | —      |

**Validation is the gate.** Nothing partially validated is executed; a task is accepted or rejected atomically.

## 4. Rationale for the key choices

### Which LLM, and why

Claude Sonnet, reached through the **Claude Agent SDK over Claude Code’s OAuth login**, so the system draws on an existing Claude subscription rather than consuming API credits. Three reasons for Claude specifically: reliable structured JSON output (7/7 first-pass valid across varied phrasings, zero repair rounds); willingness to decline rather than guess, which is the behaviour the safety argument depends on; and a company subscription already in place.

The provider is one class behind an interface (`llm_provider.py`). Three backends — `agent_sdk`, `api`, `offline` — share a `call()` signature. Swapping to a local or self-hosted model means writing one class; the validator, compiler and ROS node are untouched.

**For costing:** the business case prices the LLM at API rates even though the demo runs on a subscription. “Free because it is on a subscription” is not what per-mission cost looks like at customer scale, and the OAuth token is licensed for individual use — production would use an API key.

### Where the boundary lies

| Entrusted to the model              | Kept deterministic                     |
|-------------------------------------|----------------------------------------|
| Understanding intent from free text | Whether a named field exists           |
| Mapping “sector 2” to a field       | Where that field is, in metres         |
| Inferring a field from a crop       | Route, waypoints, speed                |
| Judging urgency                     | What urgency is allowed to change      |
| Deciding a request is too vague     | Whether the plan is safe or affordable |

The model is used for language, and only for language. Every quantity that reaches a motor is computed by code from a config file.

**One consequence, made concrete.** Urgency raises image spacing rather than speed. The model can mark a request urgent; it cannot make the robot go faster, because speed is capped in the task generator and re-checked by the validator. Going faster would blur the imagery the inspection depends on.

### Grounding rather than filtering

The prompt is assembled at runtime from the site’s own configuration, so the model is told which fields exist before it is asked to name one. This makes hallucinated identifiers rare rather than merely caught.

Field resolution then matches farmer phrasing against an alias table — "north field", "field 1", "sector 1" and "north\_field" all resolve to the same mapped field.

## 5. Failure handling

### When the LLM produces an invalid, incomplete or non-executable plan

| Failure                         | Response                                                                                                   |
|---------------------------------|------------------------------------------------------------------------------------------------------------|
| Field not on this farm          | Model returns <code>clarification_needed</code> ; nothing executes; counted as a human intervention        |
| Request too vague               | Same                                                                                                       |
| Response is not valid JSON      | Fenced blocks recovered automatically; otherwise retried once with the parse error fed back; then reported |
| Invalid crop or inspection type | Dropped and <b>reported to the farmer</b> , not silently discarded                                         |
| Plan violates a precondition    | Validator rejects with a specific, actionable code                                                         |
| Plan compiles too large         | Rejected with a suggestion to widen the capture interval                                                   |

Nothing partially validated executes. Every rejection carries a reason the operator can act on.

**Worked example — the model was willing, the validator was not.** "check everything" parsed to all three fields at confidence 0.60 and compiled to 39 waypoints using 97% of usable energy. The validator rejected it: `ENERGY_MARGIN` — the energy model is an uncalibrated estimate and leaves no room for a replan. Split this into separate missions. The mission was never published. This threshold was tightened from a warning to an error *because of* that live result.

### The eleven preconditions

`STRUCT` — waypoints present · map name matches · rows exist on this farm. `GEOMETRY` — every waypoint inside the geofence · outside every no-go zone. `SIZE` — at most 40 waypoints. `SPEED` — commanded speed within the robot cap. `STATE` — localized (`pose_covariance < 0.5`) · camera online · storage free. `ENERGY` — within budget, and under 90% of usable energy above reserve. `SAFETY` — obstacle avoidance enabled · worker distance  $\geq 2.0$  m.

Error codes: `NO_WAYPOINTS`, `MAP_MISMATCH`, `UNKNOWN_ROW`, `GEOFENCE_VIOLATION`, `NO_GO_VIOLATION`, `MISSION_TOO_LARGE`, `SPEED_LIMIT`, `NOT_LOCALIZED`, `CAMERA_DOWN`, `NO_STORAGE`, `ENERGY_BUDGET`, `ENERGY_MARGIN`, `SAFETY_DISABLED`, `WORKER_DISTANCE`. Warnings — `SPEED_QUALITY`, `ENERGY_TIGHT` — execute but are surfaced.

## 6. The safety layer

### What is forbidden

Leaving the geofence · entering a no-go zone · exceeding 0.8 m/s · coming within 3.0 m of a person above 0.2 m/s · running with obstacle avoidance disabled · starting below the 20% battery reserve · executing an unvalidated plan.

### How it is intercepted — two independent mechanisms

**Plan time.** The validator checks every waypoint against the geofence and no-go polygons before publication. A violating plan is never published.

**Run time.** The geofence and no-go polygons travel *inside* `MissionData`. `geofencing_node` monitors pose against the fence; `danger_zone_manager` rasterises the polygons into the costmap so the global planner routes around them; `obstacle_avoidance` holds veto over `/cmd_vel` and can pause a mission mid-execution.

This is defence in depth. A bug in the validator does not disable the runtime layer, and vice versa.

### Who has the final say

#### The safety layer, without exception.

The planner cannot address it, disable it, or route around it. It has no interface the LLM can reach: the model emits a parsed request, and the geofence and danger zones are attached by the compiler from site configuration, not from anything the model said. Those nodes were written and tested before the LLM existed and are unmodified by this work.

**Observed in the recorded mission:** `obstacle_avoidance` interrupted the plan nine times. Each time `mission_manager` paused, executed lateral avoidance, re-planned and resumed — with no operator input. The LLM’s plan was overridden nine times and the mission still completed 15/15.

## 7. Interfaces

| Interface                             | Type                                           | Direction   |
|---------------------------------------|------------------------------------------------|-------------|
| <code>/farm/inspection_request</code> | <code>std_msgs/String</code>                   | in          |
| <code>/farm/inspection_result</code>  | <code>std_msgs/String</code> (JSON)            | out         |
| <code>/mission_dataa</code>           | <code>navigation_interfaces/MissionData</code> | out, TRANS: |
| <code>/mission/launch</code>          | <code>LaunchCurrentMission</code> action       | out         |
| <code>/robot/get_status</code>        | <code>GetRobotStatus</code> service            | out         |
| <code>/amcl_pose</code>               | <code>PoseWithCovarianceStamped</code>         | in          |

#### Three traps in the host stack, documented

- The mission topic is `mission_dataa`**, with a double `a`, marked `#PEZZA` in `nav_nodes/utils/constants.py`. Publishing to `/mission_data` silently does nothing.
- Area polygons must be closed and have at least 4 vertices** — a rectangle needs 5 poses. `geofencing_node` rejects malformed areas with a log *warning*, so a bad geofence yields a mission that runs **with no fence at all**. There is a regression test pinning this.
- `/mission_dataa` needs `TRANSIENT_LOCAL + RELIABLE` QoS**, or `mission_manager` misses messages published before it subscribes.

## 8. Vision stage

Plant health is judged by **HSV hue clustering**, not a trained model. Vegetation pixels are segmented by saturation and hue, grouped into connected clusters, and each cluster is classified by median hue.

**Why not a trained classifier.** A model trained on real leaf photographs would have to run on Gazebo-rendered plants at 2–3 m. That domain gap is a research problem. Hue is colour physics and behaves the same on renders and photographs — there is nothing to transfer.

Measured separation across 170 clusters: anomalous clusters had median hue 38-78, healthy clusters 104-114. **No overlap.**

**The honest limit:** this detects chlorosis, not disease. Drought stress, nitrogen deficiency and infection produce similar signatures. Naming the pathogen needs a trained classifier or a multispectral sensor.